An Investigation into using LLMs for modelling outbreaks of Ebola

Introduction

In the face of disease outbreaks, reliable predictions for future dynamics of the spread of infection are key to making informed decisions. This means healthcare systems are better equipped to cope with the strain when this happens, and resources aren't wasted when and where they are less required (Gao et al., 2025).

Predictions produced by different models for the spread of infection can be weighted together to produce an ensemble, see Figure 1. It has been found that predictions from these multi-model ensembles are often more reliable and accurate (Shandross et al., 2026).

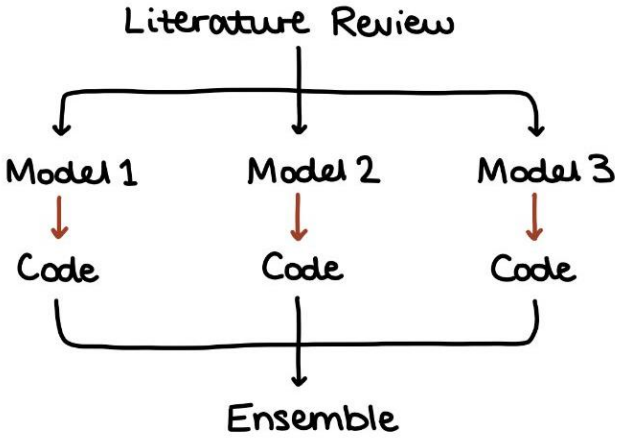


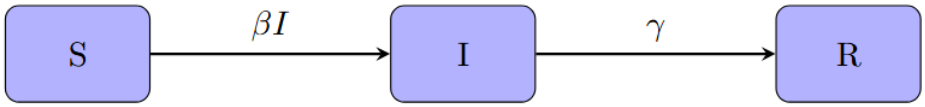
Figure 1: Producing an Ensemble

This eight-week research project focused on developing an understanding of the ability of Large Language Models (LLMs) to produce code to simulate from compartmental models, in particular for Ebola given the recent outbreak in the Democratic Republic of Congo and Uganda. Thus, we investigated the suitability of code generated from simple models and the relevance of changes made to the code to tailor the model for an Ebola outbreak. We then attached academic literature to the prompts to determine the level of consistency the generated model code had with the topics discussed in the papers.

The project has been completed in collaboration with the UKHSA, a government organisation which aims to prevent and reduce harm to people from various health threats (GOV.UK, 2021). The UKHSA make use of predictions from mathematical models to inform decisions. The rest of this case study discusses the suitability of the implementation of some LLMs to this aim.

Methods

Compartmental models are often used in epidemiology. The population is split into distinct groups, each described by an individual's current disease status. The movement of individuals between compartments is modelled by a system of ordinary differential equations (ODEs). The solutions to these equations, found via numerical methods, provide the number of individuals in each compartment at each time step. These disease trajectories are used for predictions which then help to construct informed decisions (Keeling and Rohani, 2008).



$$\frac{dS}{dt} = -\beta SI$$
$$\frac{dI}{dt} = \beta SI - \gamma I$$
$$\frac{dR}{dt} = \gamma I$$

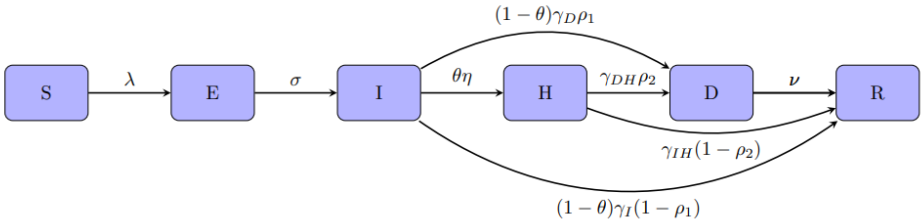
Figure 2: State transition diagram and ODEs for SIR model

The SIR (Susceptible – Infected – Recovered) model, Figure 2, splits the population into 3 compartments and is used in epidemiology for modelling the disease dynamics in this simple case. Individuals in the population are categorised as susceptible (able to be infected by the pathogen), infected or recovered (cleared the pathogen and received immunity). The system of ODEs here describes the movement from compartment S to I and from I to R, where:

- β is the rate of transmission (inverse of incubation period);
- γ is the rate of recovery (inverse of infectious period).

To construct a compartmental model specified to Ebola we need to consider the key epidemiological features of the disease.

Ebola is a highly infectious virus which is spread through direct transmission. An infected individual is not infectious immediately. We represent this incubation period, estimated at 2-21days (WHO, 2025), using an exposed compartment (E), Figure 3. Isolation in hospital is the usual treatment for Ebola, where individuals are kept hydrated and quarantined to reduce transmission. Recently deceased individuals contribute to transmission, and removal from this group is modelled by a rate of safe burial. Splitting the infectious population into the infected in community, infected in hospital, and infected deceased allows us to consider the differing transmission rates between these groups.



$$\lambda = \frac{\beta_I I + \beta_H H + \beta_D D}{N}$$

$$\frac{dS}{dt} = -\lambda S$$

$$\frac{dE}{dt} = \lambda S - \sigma E$$

$$\frac{dI}{dt} = \sigma E - \Big[\theta\,\eta + (1-\theta)\big[\gamma_I(1-\rho_1) + \gamma_D\rho_1\big]\Big] I$$

$$\frac{dH}{dt} = \theta\,\eta I - \Big[\gamma_{IH}(1-\rho_2) + \gamma_{DH}\rho_2\Big] H$$

$$\frac{dD}{dt} = (1-\theta)\gamma_D\rho_1 I + \gamma_{DH}\rho_2 H - \nu D$$

$$\frac{dR}{dt} = (1-\theta)\gamma_I(1-\rho_1) I + \gamma_{IH}(1-\rho_2) H + \nu D$$

Figure 3: State transition diagram and equations for SEIHDR model

Disease dynamics are modelled by a complicated system of ODEs, and the solution to this involves numerical approximations. In the code generation portion of this project, we have used Odin and PyGOM in R and Python respectively to find the solutions to these equations, Figure 4.

Odin is a Domain Specific Language (DSL) in R for describing and solving ODEs (GitHub, 2026). PyGOM is a package in Python for building and solving ODEs, with a focus on epidemiological models (Tye et al., 2018).

```r
library(odin)
library(ggplot2)

# Compiling the model
sir_model <- odin({
  # Differential equations
  deriv(S) <- - beta * S * I / N
  deriv(I) <- beta * S * I / N - gamma * I
  deriv(R) <- gamma * I

  # Initial conditions
  initial(S) <- S_ini
  initial(I) <- I_ini
  initial(R) <- R_ini

  # Total population
  N <- S + I + R

  # User-defined parameters
  beta <- user()
  gamma <- user()

  S_ini <- user()
  I_ini <- user()
  R_ini <- user()
})
```

```python
states = ['S', 'I', 'R']
params = ['beta', 'gamma']
transitions = [
    Transition(
        origin='S',
        equation='-beta*S*I',
        transition_type=TransitionType.ODE
    ),
    Transition(
        origin='I',
        equation='beta*S*I - gamma*I',
        transition_type=TransitionType.ODE
    ),
    Transition(
        origin='R',
        equation='gamma*I',
        transition_type=TransitionType.ODE
    )
]
sir = DeterministicOde(
    state=states,
    param=params,
    ode=transitions
```

Figure 4: SIR code using Odin (left) and PyGOM (right)

Language models and resources used

This project intended to compare multiple LLMs and interfaces they are accessed through.

We have made use of 2 Copilot interfaces: GitHub Copilot through Visual Studio Code (using an education account) and the Copilot chat function through Microsoft 365.

The Microsoft 365 Copilot gives the option for different GPT models as well as an auto function where the Copilot selects a model and length of time to think itself. Originally, the GPT models available were 5.2 or 5.5 with Quick Response or Think Deeper functions in both.

The chat function used through Visual Studio (VS) Code selected a model of its own choice, providing the model it has used at the end of each response. This was generally MAI-Code-1-Flash, Claude Haiku 4.5 or GPT 5.4 mini. The VS Code interface was particularly useful for this project's aim. The integrated chat panel and workspace made code generation and debugging more convenient. Copying code between other AI interfaces and the desired Integrated Development Environment (IDE) was much more cumbersome.

Additionally, some basic prompting was provided to an Isambard vllm to investigate the code generated by this open-source AI. The prompting was completed through a Pi assistant, and the LLM used was the Qwen 3.6 35B A3B FP8 model. We installed the pi-hermes-memory package to allow the LLM to learn from previous prompting and outputs.

Results and Discussion

We prompted the LLMs in three ways to compare success across programming languages and investigate the ability of different LLMs to incorporate other concepts into the code. First, we explored the ability of the LLMs to generate generic compartmental models, then specific models of Ebola and finally we provided literature to see how well it could re-create those models.

Generic Compartmental Models

We began by considering the consistency of the models produced here in Odin and PyGOM as well as between the 3 LLM interfaces (Microsoft 365, VS Code and Isambard).

Example Prompt: Code up a compartmental model for the disease spread of Ebola using R/Python/Odin/PyGOM

All 3 interfaces successfully generated code for SIR models. The GPT models (5.2 and 5.5) through Microsoft 365 generated Odin and PyGOM code with correct syntax and documentation. The VS Code MAI-Code-1-Flash LLM needed the prompt to specify which PyGOM classes to use, otherwise the code would call outdated ones. Further PyGOM code through the VS Code interface was written more effectively due to LLM access to the existing workspace which had correctly formed code blocks. The Isambard vllm initially uses strange syntax, not in line with current Odin or PyGOM documentation. This LLM learnt from prompts describing the issues and searched current documentation for the format desired for the code to be written in, Figure 5. The code subsequently generated consistently reproduces the desired format.

```
parameters {
  beta  :  # transmission rate
  gamma :  # recovery rate
  N     :  # total population
}

state {
  S :  # susceptible
  I :  # infected
  R :  # recovered
}

init {
  S <- N * 0.999
  I <- N * 0.001
  R <- 0
}

derivatives {
  dS <- -beta * S * I / N
  dI <-  beta * S * I / N - gamma * I
  dR <-  gamma * I
}

include "odin_base.odin"
```

```
sir_odin <- function(beta = 0.3,
                     gamma = 0.1,
                     S0 = 999000,
                     I0 = 1000,
                     t = 0:200) {

  model <- odin({
    deriv(S) <- -beta * S * I / N
    deriv(I) <-  beta * S * I / N - gamma * I
    deriv(R) <-  gamma * I

    initial(S) <- S0
    initial(I) <- I0
    initial(R) <- 0

    N <- S0 + I0

    beta  <- user(0.3)
    gamma <- user(0.1)
    S0    <- user(999000)
    I0    <- user(1000)
  })

  instance <- model$new(beta = beta, gamma = gamma, S0 = S0, I0 = I0)
  as.data.frame(instance$run(t = t))
}
```

Figure 5: Code using Odin produced by Isambard vllm, initial incorrect use (left) and corrected use (right)

Coding Models for Outbreaks of Ebola

Example prompt:

Code an SIR model using R/Python/Odin/PyGOM

-> Extend this SIR model to model key aspects of an Ebola outbreak

Prompts had to specifically mention the disease aspects desired for the model (e.g. hospitalisation, rate of burial) for the Github Copilot in VS Code (using MAI-Code-1-Flash) to code a model better tailored to Ebola than an SEIRD model. The GPT models through Microsoft 365 tended to suggest full SEIHFR models, but an SEIRD model was once suggested as a “minimal teaching model”. This simplified model may have been given due to excessive previous prompting. Code was given for plotting the trajectories within each compartment, Figure 6. The Isambard vllm required guidance for correct parameterisation of the full model.

These comments indicate the necessity of prompts in certain LLM interfaces to specify the key epidemiological features for inclusion, or the parameterisation desired for the system.

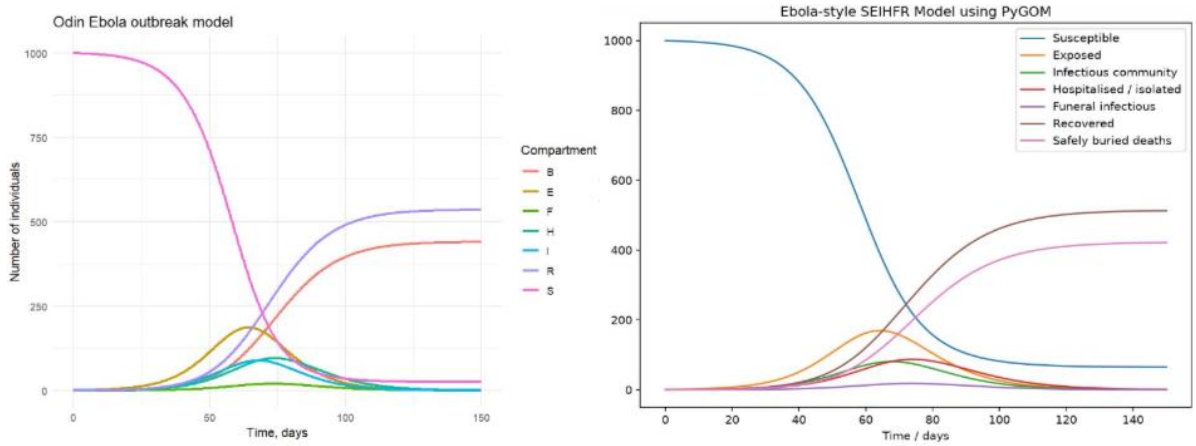


Figure 6: Trajectories from code generated using Odin (left) and PyGOM (right)

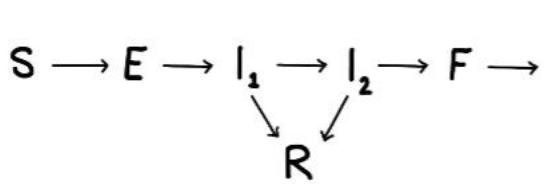
Generating Code with Attached Literature

The LLMs found through VS Code and Microsoft 365 were provided with prompts to determine whether they could successfully generate code consistent with the contents of attached literature. Prompts were given with attached literature containing varying compartment splits and areas of interest. This avenue presented the greatest difference in responses between Copilot interfaces. This may be due to strength of model used or blocks put in place for the Copilot run through VS Code.

Example prompt: Code up a compartmental model for the disease spread of Ebola using Odin (with a pdf or link to the paper attached)

<u>Copilot models through Microsoft 365:</u>

The LLMs here followed the correct model structure, Figure 7, and mentioned the key aspects of paper. The responses from the Think Deeper models had the most detail provided regarding extensions detailed in the paper. The Quick Response models were more likely to only provide code necessary for modelling the compartmental model.




```
## Core transmission model
deriv(S)  <- -lambda * S
deriv(E)  <-  lambda * S - alpha * E
deriv(I1) <-  alpha * E - gamma * I1 - r1 * I1
deriv(I2) <-  gamma * I1 - delta * I2 - r2 * I2
deriv(F)  <-  delta * I2 - delta2 * F
deriv(R)  <-  r1 * I1 + r2 * I2
```

Figure 7: Structure of a compartmental model in academic literature, (D'Silva and Eisenberg, 2017), (left) and correct implementation of this using Odin provided by GPT 5.5 Think Deeper LLM (right)

<u>Copilot models through VS Code:</u>

The Copilot models accessed through VS Code required much more explicit prompting to generate code even vaguely consistent with the attached literature. These models had a preference to build models based only on the current workspace. Additional prompts could be given to lead to the LLM to access the literature and generate more consistent code. This was implemented in future prompts.

An unsuccessful prompt, for Claude Haiku 4.5, was “Generate code using Odin to model an outbreak of Ebola, using the compartmental model given in this paper. Match the exact notation from the paper.”. This generated code for an SEIHRDB model disregarding the paper’s contents. Additionally, the content was described as taken from a hallucinated paper.

Since, in comparison, even the GPT 5.2 models on the Microsoft 365 Copilot were able to complete these prompts, it suggests that either it is the mini version that is limiting this response, or restrictions from the VS Code interface.

Challenges and Conclusions

This project aimed to investigate the ability of different LLMs to generate compartmental model code for outbreaks of Ebola. The generation of generic compartmental model code was performed correctly by all 3 AI interfaces used. Expansion of models to consider the dynamics of an Ebola outbreak was executed most successfully by the Microsoft 365 Copilot models. Approaches to producing code in line with academic literature was most varied, most likely due to interface specific restrictions.

The comments and warnings provided during prompting on the VS Code interface indicate the presence of barriers limiting the output of information matching online content and conveying too much information about sensitive topics.
Comments include:

- “Response cleared due to possible match to public code, retrying with modified prompt.”
- Invalid prompt: we've limited access to this content for safety reasons. This type of information may be used to benefit or to harm people.

These barriers limit the applicability of generating compartmental model code using academic literature at a large scale.

Further analysis of output from the Copilot through VS Code was limited due to low AI credit availability from the education account. This introduces the balance to be made between ease of interface use, AI credit cost and accessibility of information.

The Isambard vllm was used to generate compartmental model code and showed great resilience when aiming to learn new syntax. This often came at the cost of long response times where at times the model seemed stuck in loops. This open-source LLM was able to

generate correct code using both Odin and PyGOM, as well as successfully transform basic models to suit an Ebola outbreak when given suitable prompting for style of parameterisation.

The responses through VS Code and the Isambard vllm were more consistent than through Microsoft 365, due to the influence of code in the current workspace and the memory package respectively.
On the other hand, the Copilot models through Microsoft 365 were more consistently able to provide compartmental model code in line with the academic papers of interest. The outputs did not seem hindered by restrictions. However, we observed that the LLMs sometimes provided simplified versions of the code as "teaching models". This may have been an attempt to decrease the time required to generate output or may have been due to the extensive amount of prompting within the current chat.

Despite the challenges outlined, the LLMs frequently provided useful insights into the mathematical modelling aspects of the project and considered the implications of epidemiological features specific to Ebola. Further prompting to rectify issues often led to more appropriate code and improved formatting. Effective use of LLMs for this aim requires an understanding of the underlying compartmental models as well as the structure and purpose of the code. LLMs can then be viewed as a tool in the process of producing efficient and effective decisions during disease outbreaks. Consequently, a key limitation to the use of LLMs in this context is differing accessibility due to AI and healthcare restrictions.

References:

D'Silva, J.P. and Eisenberg, M.C. (2017). Modeling spatial invasion of Ebola in West Africa. *Journal of Theoretical Biology*, 428, pp.65–75. doi:10.1016/j.jtbi.2017.05.034.

Gao, S., Chakraborty, A.K., Greiner, R., Lewis, M.A. and Wang, H. (2025). Early detection of disease outbreaks and non-outbreaks using incidence data: A framework using feature-based time series classification and machine learning. *PLOS Computational Biology*, 21(2), p.e1012782. doi:10.1371/journal.pcbi.1012782.

Github.io. (2026). *1 Getting started with odin – Odin and Monty*. [online] Available at: https://mrc-ide.github.io/odin-monty/odin.html [Accessed 5 Aug. 2026].

LEGRAND, J., GRAIS, R.F., BOELLE, P.Y., VALLERON, A.J. and FLAHAULT, A. (2006). Understanding the dynamics of Ebola epidemics. *Epidemiology and Infection*, [online] 135(4), pp.610–621. doi:10.1017/s0950268806007217.

Matthew James Keeling and Rohani, P. (2008). *Modeling infectious diseases in humans and animals*. Princeton: Princeton University Press.

Shandross, L., Howerton, E., Contamin, L., Hochheiser, H., Krystalli, A., Consortium of Infectious Disease Modeling Hubs, Reich, N.G., Ray, E.L., Castro Rivadeneira, A.J., Contamin, L., Funk, S., Gerding, A., Gruson, H., Hochheiser, H., Howerton, E., Kerr, M., Krystalli, A., Loo, S.L., Ray, E.L., Reich, N.G., Sato, K., Shandross, L., Sherratt, K., Truelove, S. and Zorn, M. (2026). Multi-Model Ensembles in Infectious Disease and Public Health: Methods, Interpretation, and Implementation in R. *Statistics in Medicine*, [online] 45(1–2), p.e70333. doi:10.1002/sim.70333.

Tye, E., Finnie, T., Hall, I. and Leach, S. (2018). *PyGOM - A Python Package for Simplifying Modelling with Systems of Ordinary Differential Equations*. [online] arXiv.org. Available at: https://arxiv.org/abs/1803.06934 [Accessed 6 Aug. 2026].

UK Health Security Agency (2021). *UK Health Security Agency*. [online] *GOV.UK*. Available at: https://www.gov.uk/government/organisations/uk-health-security-agency [Accessed 5 Aug. 2026].

World Health Organization (2025). *Ebola Disease*. [online] *Who.int*. World Health Organization: WHO. Available at: https://www.who.int/news-room/fact-sheets/detail/ebola-disease [Accessed 5 Aug. 2026].